

Resilient Raspberry Pi OS – der „unkaputtbare“ Raspi



TÜBIX 2026/04. Juli 2026
Stefan Baur, X2Go Lead Evangelist

Vorstellung Stefan Baur



ct magazin für
computer
technik

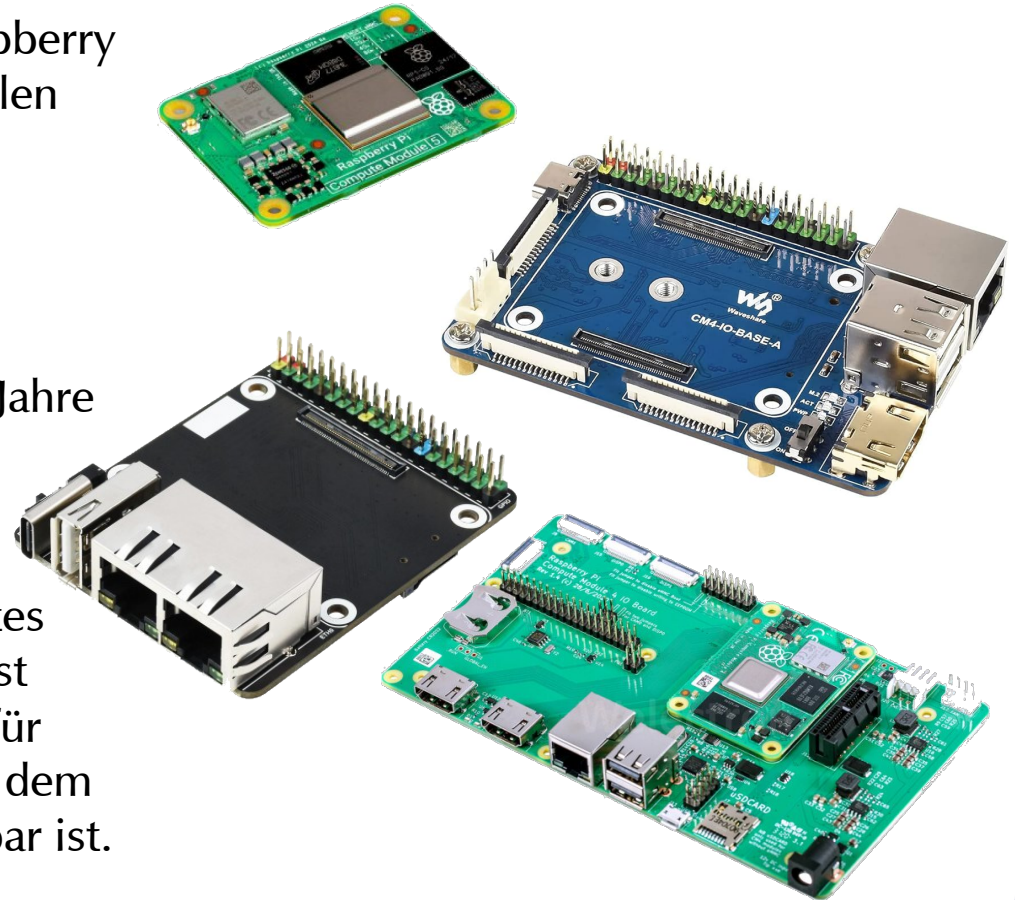
heise online **Make:**

„Volksmeinung“ über Raspis

- Nettes Spielzeug, aber nicht für professionellen Einsatz geeignet
- Zu frickelig/zu instabil:
 - Bei auch nur halbwegs anspruchsvoller Nutzung gehen die microSD-Karten ständig kaputt, außerdem sind Zugriffe träge
 - Software hängt sich gefühlt „spontan“ auf, man muss ans Gerät und es resetten
- Entsprechend unwirtschaftlich hoher Wartungsaufwand

Wie Rio Reiser singt: „Alles Lüge!“

- Zum einen gibt es spezielle Hardware der Raspberry Pi Foundation, die explizit für den professionellen Einsatz gedacht ist – die Compute Modules:
- Vorteile:
 - Auf Dauereinsatz ausgelegt
 - Ersatzteil-Verfügbarkeit auf mindestens 10 Jahre ab Markteinführung garantiert
- Nachteil:
 - Compute Modules brauchen ein sogenanntes Base-Board – das muss man entweder selbst entwickeln oder hoffen, dass es schon ein für den geplanten Einsatzzweck passendes auf dem Markt gibt, und das auch lang genug lieferbar ist.



(Neudeutsch auch: „Fake News!“)

- Zum anderen sind tatsächlich auch normale Raspberry Pis heute in diversen kommerziellen Produkten zu finden
- Erkennbar an der charakteristischen Anordnung der Ports, selbst wenn manche Hersteller es nicht offiziell zugeben
- Mit der Erfahrung aus dem Hobbybastel-Raspi-Bereich sollte man meinen, dass die enorm hohe Rückläuferquoten haben müssten. Was machen die also anders?



Kommerzielles LoRaWAN-Gateway



Rückseite eines LaserCutters



KVM-over-IP: piKVM

Die Software-Trickkiste

- Man kann softwareseitig einige Maßnahmen ergreifen, um Schreibzugriffe auf die microSD-Karte zu reduzieren:
- Logging reduzieren/in RAMDisk legen/ausschalten
- Alle, oder zumindest die meisten Schreibzugriffe in eine RAMDisk umleiten lassen → führt auch dazu, dass bei Stromausfall/hartem Reset das darunterliegende Dateisystem nicht kaputtgehen kann, weil es nur readonly gemountet ist
- Speichermedium nicht komplett partitionieren, freien Platz am Ende lassen (bis zu 20% heutzutage realistisch) → der Controller bekommt einen größeren Pool an nicht genutzten Speicherzellen

Die Hardware-Trickkiste

- Leider sind die wenigsten Controller auf microSDs zu einem vernünftigen Wear-Leveling (also der Rotation häufig beschriebene Zelle → selten beschriebene Zelle) in der Lage, daher kommen die häufigen Ausfälle
- Auf „größeren“ Medien wie USB-Sticks, SSDs, NVMe und eMMC sitzen aber meist bessere Controller → meist günstiger als irgendwelche „commercial grade microSDs“ zu haben, trotz vergleichbarer Qualität
- Alle Pis ab Pi3B sind auch von USB bootfähig! Pi4 natürlich auch von USB3, für den Pi5 gibt es sogar nativen NVMe-über-PCIe-Support (Zusatzplatine notwendig), die Compute Modules gibt es mit Onboard-eMMC (was auch für industriellen Einsatz ausgelegt ist)
- Vorgängermodelle Pi1/Pi2 brauchen (micro)SD zum Booten, können aber das Rootsystem von einem USB-Medium laden → auch so kann man die meisten Schreibzugriffe umleiten

Mit der Physik verhandelt man nicht.

- Gilt nicht nur bei der Klimaerwärmung und beim Auto- und Zweiradfahren, sondern auch bei Massenspeichern – also von microSD über USB-Sticks bis zur NVMe-SSD
- Speicherchips produzieren Abwärme:
 - Die Abwärme ist um so größer, je mehr/je länger am Stück geschrieben wird (Lesezugriffe erzeugen auch Wärme, aber nicht so viel)
 - Je wärmer der Chip wird, desto schneller altert er
- Controller fragen daher die Chiptemperatur ab und drosseln die Zugriffsgeschwindigkeit, wenn die Temperatur steigt → System wird träge
- Je weniger Chips auf dem Medium verbaut sind, und umso kleiner sie sind, umso größer das Temperaturproblem:
 - Geringere Abstrahlfläche → man wird die Hitze nicht so schnell los und altert den Chip
 - Mit mehreren Chips könnte man durchrotieren, einer wird beschrieben, der andere darf abkühlen → hat man nur einen, geht das nicht

Cool McCool

- Nicht nur der Speicher, auch die CPU, GPU, etc. auf dem Pi selbst freuen sich über Kühlung
- Hitzestau führt erst zu Drosselung, dann zum harten Not-Aus
- Unbedingt Kühlkörper aufkleben
- Möglichst große Kühlkörper wählen (besonders bei passiver Kühlung):
 - Es gibt z.B. das Gehäuse „Argon One“, bei dem das ganze Gehäuse als Kühlkörper dient
 - Lüfter sind mechanisch bewegte Komponenten und fallen daher gern vorzeitig aus:
 - Besser, wenn man den Lüfter nie oder nur als Reserve bei einer Hitzewelle braucht
 - Lüfter verwenden, deren Drehzahl man über GPIO bis 0 herunterregeln kann
- Hardware-Watchdog des Raspberry Pis aktivieren! → Löst automatischen Reset aus, wenn der „Heartbeat“ des Betriebssystems ausbleibt.

Sparfuchs-Ansatz

- PCIe-NVMe + Kühlkörper oder USB-SSD sind die einfachsten Lösungen, aber auch die teuersten
- Die eMMC-Option hat man nur bei den Compute Modules
- USB-SSD ist ggf. auch arg sperrig, je nach Bauform
- Wir nehmen also handelsübliche USB3-Sticks, aber:
 - Mit hohen Transferraten
 - Mit mehreren Chips
 - Mit großem Gehäuse, und das möglichst aus Metall:
 - Bessere Wärmeabfuhr
 - Nachrüsten von zusätzlichem Kühlkörper möglich (Selbstklebe-Wärmeleitpad und z.B. entsprechend geformtes Kupferblech oder Aluminium-Rippenkühlkörper)

Praxiserfahrungen USB-Sticks

- Bei uns haben sich bisher gut bewährt:
 - Kingston DataTraveler Max Type-A, 1000MB/s Lese-/900MB/s Schreibrate laut Datenblatt
 - Kein Metallgehäuse → daher kein Kühlkörper möglich
 - Aber groß und verteilt Zugriffe auf mehrere Chips
 - Sandisk Extreme Pro (in der Variante mit Metallgehäuse und mittels Schieber versenkbarem USB-Port), 420MB/s Lese-/380MB/s Schreibrate laut Datenblatt
 - Metallgehäuse hat ovale Form → kein Standard-Kühlkörper möglich, man muss basteln
 - Mit Einschränkungen empfohlen, weil das smartctl-Tool nach kurzer Zeit meldet, der Speicher sei kurz vor dem Defekt und man solle ihn austauschen.
→ Tatsächlich aber bis heute kein Datenverlust, evtl. Fehlalarm, aber nichts für schwache Nerven.
 - SanDisk Ultra Luxe, 150MB/s Leserate, keine offizielle Angabe zur Schreibrate, vermutlich 30-60 MB/s
 - Metallgehäuse, nahezu plane Form, gut geeignet für zusätzlichen Kühlkörper

Zwischenfazit

- Compute Module: Ein Modell mit onboard-eMMC verwenden, das ist drauf ausgelegt. Nicht den ganzen Speicher partitionieren schadet trotzdem nicht, die Schreibzugriffe auf die RAMDisk umzuleiten, wäre hier rein aus Lebenszeitgesichtspunkten verzichtbar (aber nicht, was Stromausfall/hartes Ausschalten angeht) → insgesamt eher Kategorie „Fire & Forget“ – läuft einfach.
- „Normaler“ Raspberry Pi 3B, 3B+, 4B oder 5: Ab Pi4 USB3.x-Stick verwenden, Hardware wie vorhin beschrieben auswählen, Platzreserve bei der Partitionierung vorsehen, damit der Controller Manövriermasse hat, Schreibzugriffe auf RAMDisk umleiten
- Kühlkonzept und Hardware-Watchdog nicht vergessen!
- Man könnte auch noch eine „StromPi“-Zusatzplatine mit Akku anbringen, wenn man sich seine eigene kleine USV basteln will (und im Gehäuse noch genug Platz ist).

Und nun?

- Jetzt haben wir eine Basis, die schon recht „unkaputtbar“ ist, was äußere Einflüsse wie Temperatur und Stromausfall angeht – mit einem StromPi + Akku laufen wir sogar während kurzer Stromausfälle weiter, aber auch ohne kommen wir wieder automatisch hoch, sobald der Strom wieder da ist.
- Aber wie lösen wir das Problem, dass wir auch Updates einspielen müssen? „Draußen“, also im Feld, im Einsatz, irgendwo in der hintersten Ecke verbaut, wo man nicht mal schnell den Stecker ziehen und die microSD tauschen kann?

Nobody ever got fired for trusting IBM

- IBM hatte schon in den 1990er-Jahren in einem sogenannten „Redbook“ ein Betriebskonzept für IBM OS/2 WARP3 herausgebracht, welches mehrere Betriebssystem-Partitionen vorsah, zwischen denen mittels Bootloader durchgewechselt werden kann:
 - Wartungspartition (nur Bootloader und ein minimales Toolset)
 - Produktionsumgebung 1
 - Produktionsumgebung 2
- Updates werden im Wechsel eingetankt und dann der Default-Booteintrag geändert
- Auf PCs spart man sich das heutzutage meistens, weil man die Installationsumgebung auch übers Netzwerk in eine RAMDisk booten kann, von dort vollen Zugriff auf das Massenspeichermedium hat, und dann einfach alles platt macht und neu installiert.

Zurück in die Vergangenheit

- Stellt sich raus: Dieses Konzept mit mehreren Bootumgebungen wird von der Raspberry Pi Foundation ganz offiziell auf dem Raspberry Pi unterstützt – man braucht kein alternatives Image wie NOOBS oder PINN, keine Zusatzsoftware, keinen speziellen Bootloader wie uboot oder so.
- Nur: Die Standardinstallation von Raspberry Pi OS mittels rpi-Imager bzw. dd an der Kommandozeile schießt sich dabei selbst ins Bein, weil sie keinen freien Platz für zusätzliche Umgebungen lässt. Selbst wenn man das Flag setzt, dass die Root-Partition nicht den gesamten freien Speicherplatz in Anspruch nehmen soll – die Partitionstabelle passt nicht.
- Um das zu verhindern, dürfen wir den Raspi nicht direkt von dem frisch betankten Medium booten, sondern müssen erst noch am PC ein paar Anpassungsschritte vornehmen. Das sind immer die gleichen, egal was man danach installieren will, deswegen bietet es sich an, das in ein Skript zu packen, und das als Basis für andere in einem öffentlichen Git-Repo bereitzustellen. Genau das habe ich getan. :)

Aufbau der Umgebung

- Die drei Umgebungen heißen hier ENV1, ENV2 und ENV3.
- Wie zu IBM-OS/2-Zeiten ist die erste Umgebung die, die nur das absolute Minimalsystem enthält.
- Die Partitionierung sieht nun so aus:
 - Partition 1 (primäre): /boot von ENV1, FAT32, 512 MB groß
 - Partition 2 (primäre): /boot von ENV2, FAT32, 512 MB groß
 - Partition 3 (primäre): /boot von ENV3, FAT32, 512 MB groß
 - Partition 4: (erweiterte):
 - Partition 5 (logische): / von ENV1, EXT4, 8 GB groß
 - Partition 6 (logische): / von ENV2, EXT4, 8 GB groß
 - Partition 7 (logische): / von ENV3, EXT4, 8 GB groß
 - Partition 8 (logische): /data – wird in jedem ENV gemountet, EXT4, restlicher freier Speicherplatz (Diese Partition kann man bei Bedarf aus ENV1 heraus aushängen und schrumpfen, um z.B. 20% als Reserve/Manövriermasse für den Controller zu lassen)

Besonderheit: Netzwerkdevice

- Das Netzwerkdevice ist hier, abweichend von der Raspberry-Pi-OS-StandardEinstellung die Bridge br0
- Das Device eth0 ist automatisch in der Bridge, so dass sich das System erst mal wie gewohnt verhält
- Weitere Devices, die auf eth* matchen, werden in manchen Branches (single-port-printserver, multi-port-printserver) sogar schon am Ende der Installation automatisch in die Bridge br0 geworfen, um zu zeigen, dass und wie das geht.
- Warum habe ich das so gemacht?
 - Weil es Compute-Module-Baseboards mit mehr als einem Ethernet-Port gibt, und auch USB-Ethernet-Dongles existieren.
 - Auf diese Weise kann man die anderen Ports bequem in die Bridge aufnehmen, oder in weitere Bridges, oder einzeln nutzen.
 - Damit kann man z.B. einen Wifi-Access-Point bauen, den man einfach bei einem bestehenden Gerät mit LAN-Kabel (z.B. einem IP-Telefon oder PC) „dazwischenstecken“ kann, so dass man keinen zusätzlichen Switch-Port belegt.

Github-Repository und Branches

- Repository-URL: <https://github.com/stefanbaur/rrpi-base-install>
- Spezielle Branches:
 - main – Alle Branches holen sich Branch-übergreifende Weiterentwicklungen per merge von hier. Dieser Branch lässt sich daher als einziger nicht installieren.
 - example – ein Minimalbeispiel, welches nur das Programm „sl“ installiert („apt install -y sl“). Sehr simpel gehalten und erleichtert daher den Einstieg, eigene Branches aufzusetzen.
 - minimal – im Prinzip wie der example-Branch, nur ohne „sl“ – das eignet sich für eine Basisinstallation, die anschließend mit Orchestrierungstools wie ansible verwaltet wird

Direkt nutzbare Branches

- Folgende Branches lassen sich direkt nutzen:
 - x2goserver-kdrive-lxde – eine kleine X2GoServer-Demo-Instanz mit X2Go-KDrive, LXDE, und zwei Beispielaccounts mit SSH-Keys
 - x2gominiclient – ein minimalistischer Desktop mit einem X2GoClient, passend zum x2goserver-Branch vorinstalliert
 - single-port-printserver – installiert einen Daemon, der alle per IP auf Port 9100 eingehenden Daten auf einen per USB angeschlossenen Drucker wirft. Kann man auf alten Pi1/Pi2 nutzen, um sie noch sinnvoll weiterzuverwenden.
 - multi-port-printserver – Erweiterte Version des obigen Branches, hier können alle USB-Ports des Pis genutzt werden, Ports 9100, 9101, usw. werden immer auf den gleichen USB-Port und somit auch dann auf den gleichen Drucker gemappt, wenn mehrere Drucker angeschlossen sind, aber ggf. nicht alle gleichzeitig eingeschaltet wurden.
 - docker-mailcow – installiert eine komplette dockerisierte Mailcow-Instanz

Weitere Branches

- Work in Progress:
 - grannophone – hier findet die Software-Entwicklung für mein Senioren-Videotelefon „Grannophone“ statt
 - lan-2-upstream-wifi-bridge – dockt sich an ein vorhandenes WLAN an, und ermöglicht, Geräte, die nur LAN- oder USB-Ports haben, ins WLAN einzuklinken
- Geplant:
 - wifi-accesspoint – spannt ein WLAN auf, mit dem man sich dann Richtung LAN verbinden kann
 - nas-proxy – dazu gedacht, Home-NAS-Systeme abzukapseln, wenn der Hersteller keine Firmware-Updates mehr bereitstellt bzw. die Firmware „fishy“ Dinge tun versucht (I’m looking at you, Western Digital – heimlich einen VPN-Tunnel zur Herstellerfirma aufbauen, obwohl die Funktion in der WebGUI vom Besitzer abgeschaltet wurde, ist nicht cool!)

Wie setzt man das System nun auf?

- Für die Grundinstallation, so wie sie auch von der Raspberry Pi Foundation vorgesehen ist, gibt es mittlerweile im Git-Repo <https://github.com/stefanbaur/rrpi-base-install/> im Ordner `rpi-imager-2.0.x-manual/de` eine bebilderte, deutschsprachige Schritt-für-Schritt-Anleitung von mir – und ich unterstelle mal, dass ihr den rpi-imager alleine installiert bekommt. Tipps dazu stehen in der README.md im obersten Verzeichnis meines Repos.
- Wer von euch den Begriff cloud-init schon mal gehört hat: Das Image unterstützt das mittlerweile ebenfalls, und der rpi-imager nutzt es auch. Ihr könnt den rpi-imager also umgehen, und eure eigenen cloud-init-Konfigurationsdateien in das Image werfen, wenn ihr euch damit auskennt.

Wichtige Hinweise

- Wenn ihr ein Compute Module mit eMMC verwendet, braucht ihr ein Zusatzprogramm, welches dem PC vorgaukelt, das CM sei ein USB-Stick. Was ihr dafür installieren müsst, und wie das genau funktioniert, steht ebenfalls in der README.md im obersten Verzeichnis meines Repos.
- Dem Imager ist es vollkommen egal, ob er eine (micro)SD-Karte, ein USB-Speichermedium wie einen Stick oder eine USB-SSD, oder ein CM, welches sich als Stick ausgibt, vorgesetzt bekommt. Er macht keinen Unterschied und betankt alles gleich.
- Bei Schritt 9 in der Anleitung, wenn es darum geht, den Benutzernamen setzen → so ihr später Ansible zur Orchestrierung verwenden wollt, weil ihr ein ganzes Rudel dieser Pis aufzusetzen plant, dann hier den Namen eintragen, den euer Ansible-Controller verwendet, um SSH-Verbindungen aufzubauen (dafür scheint sich „vagrant“ eingebürgert zu haben, aber das kann bei euch ja durchaus abweichen). In Schritt 12 bis Schritt 14 geht es dann darum, wie ihr den passenden SSH-Public-Key einbindet.
- Wenn der Imager euch erzählt, er ist fertig, auf keinen Fall den Pi mit diesem Medium booten! Ihr müsst die nächsten Schritte noch am PC mit dem kurz abgezogenen und dann wieder neu eingesteckten USB-Stick (beim CM daran denken, auch das Zusatzprogramm neu zu starten) durchführen, erst danach dürft ihr den Pi davon booten!

Die Skriptkonfiguration

- Wir klonen das Repository und checken erst mal den main-Branch aus:

```
mkdir -p ~/git && cd ~/git
git clone https://github.com/stefanbaur/rrpi-base-install.git
cd rrpi-base-install
git branch -a                # → zeigt die Liste aller verfügbaren Branches
git checkout main            # den main-Branch auswählen
```

- Die für alle Branches einheitliche „Magie“ steckt in der Datei „base_install.conf“. Wenn ihr euch keinen eigenen Branch anlegen wollt, könnt ihr sie in eine Datei „base_install_custom.conf“ kopieren, diese hat immer Vorrang vor der „base_install.conf“ aus dem Repo und bleibt beim Branch-Wechsel stehen.
- Was gleich wichtig wird: Ihr braucht irgendwo ein temporäres Verzeichnis mit mindestens 15GB freiem Speicherplatz.
→ Voreingestellt ist „/tmp“, aber das lässt sich ändern.

base_install.conf – 1/3

```
# Anpassen, wenn /tmp bei euch keine 15GB frei hat
export TEMPIMAGEDIRROOT="/tmp"
```

```
# Auf "true" setzen, wenn ihr einen Pi4 oder Pi5 habt
# und noch am Testen/Debuggen seid:
export DEBUG_VIA_USBETH="false"
# bei "true" müsst ihr auch
export FIRSTTHREEOCTETS="10.0.0" # und
export NETMASK="255.255.0.0"      # setzen
```

```
# in der Folge wird auf dem Pi dann ein DHCP-Server aktiviert, der aber nur auf dem
# USB-C-Port greift, an dem man das Netzteil anschließen würde. Steckt ihr dort ein
# USB-C-to-USB-A oder USB-C-to-USB-C-Kabel ein, welches ihr mit dem anderen Ende in
# euren PC steckt, holt sich der Pi den Strom vom PC, präsentiert sich dem PC als
# USB-Netzwerkadapter, und vergibt ihm eine IP (wenn ihr das obige Beispiel
# übernehmt) zwischen 10.0.0.10 und 10.0.0.20, so dass ihr per ssh username@10.0.0.1
# auf den Pi zugreifen könnt – auch ohne eingestecktes Netzkabel und ohne
# konfiguriertes WLAN.
```

base_install.conf – 2/3

```
# Auf "true" setzen, wenn ihr ein SPI-Display
# (kreditkartengroßer TFT zum Aufstecken auf die GPIO-Ports) habt
# und noch am Testen/Debuggen seid:
export DEBUG_VIA_SPI="false"

# Damit werden alle Ausgaben, die normalerweise auf einem per HDMI
# angeschlossenen TFT erscheinen würden, auf das SPI-Display
# ausgegeben. Somit braucht ihr dann nur noch eine USB-Tastatur, um
# euch lokal einloggen zu können. Das ist praktisch, wenn ihr euch
# z.B. irgendwelche Netzwerkprobleme eingefangen habt.
```

Der Clou: Hardware-Boot-Umschalter

- Mit einem GPIO-Pin, den man in der `base_install.conf` definieren kann, kann man den Raspberry Pi dazu zwingen, beim nächsten Neustart das ENV1 zu booten – selbst wenn in der Konfigurationsdatei des Bootloaders etwas anderes steht. Dazu muss man ihn nur beim Neustart auf Masse ziehen.
(Hinweis: Es ist nicht sauber dokumentiert, wie lange der Pin auf Masse bleiben muss. Am besten abwarten, bis die ACT-LED ein Herzschlag-Blinken zeigt.)
- Der Fantasie sind da praktisch keine Grenzen gesetzt, vom einem Jumper, den man bei Bedarf aufsteckt, über einen normalen Schalter/Taster, einen versenkten Taster, den man nur mit einer aufgebogenen Büroklammer erreicht, bis zum Schlüsselschalter ist alles drin. Auch ein Schalter mit einem Seilzug, wie bei einer altmodischen Tisch- oder Stehlampe wäre denkbar, um den Reset bei einem nur umständlich erreichbar verbauten Raspi bequem aus der Ferne auszulösen, oder ein Fotosensor in Verbindung mit einer Glasfaser, falls man Streckenlängen überbrücken muss, bei denen man mit einem Kabel kein zuverlässiges Signal mehr hinbekommt, bzw. wo Überspannungsgefahr bei Blitzschlag droht (Raspi auf der Kirchturmspitze, der mit der Webcam das Storchennest filmt und per WLAN streamt).
- Da in ENV1 nur die Minimalinstallation steckt, und man es daher selten updaten muss, stehen die Chancen gut, hier noch auf eine bootfähige und per SSH aus der Ferne erreichbare Umgebung zu treffen, wenn man sich die andere(n) zerlegt hat.

base_install.conf – 3/3

```
# Nummer des GPIO-Pins, den man auf Masse ziehen muss, damit beim
# nächsten Start zwingend ENV1 gebootet wird, egal, was in der
# Boot-Datei steht. Zum Deaktivieren des Features leer lassen.
export GPIO_FORCE_ENV1="4"
```

```
# Wenn man den Raspi so hinlegt, dass die beiden GPIO-Pins mit den
# +5V oben rechts liegen, dann ist GPIO 4 der dritte Pin von oben
# auf der linken Seite, und als Masse-Pin eignet sich der vierte
# Pin von oben auf der rechten Seite.
```

```
# Alternativen:
```

```
# * GPIO 15 – fünfter Pin von oben auf der rechten Seite,
#             Masse fünfter Pin von oben auf der linken Seite
```

```
# * GPIO 21 – Der Pin ganz unten rechts,
#             Masse der Pin ganz unten links
```

„example“-Branch

- Wir wechseln für unser erstes Beispiel in den Branch „example“:
`git checkout example` # den example-Branch auswählen
- Hier schauen wir uns die Shellskripte im Verzeichnis „./templates“ an:
`cd templates`
`ls -l *.sh`

- Dort befinden sich:

`autostart.sh` # wird bei der Installation in jedem ENV und bei jedem Start
ausgeführt – *kürzt sich aber nach der Installation selbst zusammen*

`ENV1-custom-install.sh` # Wird bei der Installation in ENV1 ausgeführt, *WIRKT*
aber auch in ENV2 und ENV3 – da diese aus ENV1 geklont werden

`ENV2-custom-install.sh` # wird nach dem Klonvorgang nur in ENV2 ausgeführt

`ENV3-custom-install.sh` # wird nach dem Klonvorgang nur in ENV3 ausgeführt

- Wiederum: Wollt ihr nicht euren eigenen Branch pflegen, könnt ihr eure Änderungen statt in „./templates“ einfach in „./custom“ ablegen – dort haben sie Vorrang vor den Templates und werden nicht von einem `git pull` überschrieben.

ENV1-custom-install.sh

- Im „example“-Branch sieht das recht einfach aus.
- Ein -e führt dazu, dass bei Fehlern abgebrochen wird, so dass wir keine „verbogene“ Installation bekommen, sondern sauber auf einen Fehler laufen.
`#!/bin/bash -e`
- Ein paar erklärende Kommentare, was man hier warum tun bzw. besser lassen lassen sollte.
`# THIS SCRIPT WILL BE EXECUTED INSIDE THE CHANGEROOT, NO NEED TO CALL chroot HERE`
`# run this right before umounting, so there's no apt clean call destroying our work`
- Hier nun der Aufruf von `apt-get install`, mit einem `echo` darüber, damit man auch sieht, wo das Skript gerade ist, und warum es diese Pakete herunterladen muss (an dieser Stelle werden sie durch das -d nur heruntergeladen, aber noch nicht installiert) – sie sind nämlich Voraussetzung für ein funktionierendes Overlay-Dateisystem. Dieses aktivieren wir später für die Root-Partitionen von ENV1, ENV2, und ENV3 – nur /data bleibt verschont.
WICHTIG: Wie schon auf der vorangegangenen Folie erklärt, werden alle Pakete, die man hier auflistet, hinterher auch in ENV2 und ENV3 zu finden sein, und alle Schritte, die man hier sonst noch ausführen lässt, werden auch in ENV2 und ENV3 Auswirkung haben.
`# download the packages required for overlayroot, so you don't need internet access to activate it`
`echo "Downloading required packages for overlayroot, so it can be installed later - even without internet access."`
`apt-get install -d -y cryptsetup cryptsetup-bin overlayroot`

ENV2/ENV3-custom-install.sh

- Die Skripte für ENV2 und ENV3 unterscheiden sich deutlich von dem von ENV1, aber praktisch kaum voneinander.
- Ein -e führt wieder dazu, dass bei Fehlern abgebrochen wird, so dass wir keine „verbogene“ Installation bekommen, sondern sauber auf einen Fehler laufen.

```
#!/bin/bash -e
```

- Auch hier gibt es wieder den erklärenden Kommentar, was man hier besser lassen sollte.
THIS SCRIPT WILL BE EXECUTED INSIDE THE CHANGER00T, NO NEED TO CALL chroot HERE
- Und wieder der Aufruf von apt-get install, mit einem echo darüber, damit man auch sieht, wo das Skript gerade ist, und welche Pakete es zur Installation vorbereiten will (auch an dieser Stelle werden sie durch das -d nur heruntergeladen, aber noch nicht installiert).

Da wir im „example“-Branch sind, ist es nur ein Paket, nämlich „sl“ – die Steam Locomotive.

In diesem Branch unterscheiden sich die beiden Dateien nur dadurch, dass in ENV3-custom-install.sh statt dem Text „ENV2“ dann „ENV3“ steht.

Natürlich wäre es aber möglich, ENV2 und ENV3 an dieser Stelle mit unterschiedlichen Paketen zu betanken, z.B. weil man ENV3 mit zusätzlichen Debug-Tools versehen will.

```
echo "Downloading the 'sl' package into ENV2, so it can be installed later - even  
without internet access."
```

```
apt-get install -d -y sl
```

autostart.sh – 1/2

- Die `autostart.sh` wird nach `/data` kopiert und von dort ausgeführt. Das mag riskant erscheinen, aber da wir `/data` im Normalfall mit „`sync`“ mounten, ist auch hier das Risiko eines Datenverlusts durch ein kaputtes Dateisystem bei plötzlichem Crash oder Stromausfall gering.
- Diese Datei ist so groß, dass wir sie im Rahmen dieses Vortrags nicht sinnvoll im Detail durchgehen können, daher nur zwei Folien dazu.
- Während der Erstinstallation/-konfiguration sorgt sie dafür, dass die vom `rpi-imager/cloud-init` gesetzten Änderungen auch wirklich eingespielt sind, bevor unsere Anpassungen laufen – anschließend löscht sie das `cloud-init` Paket.
- Außerdem ist in ihr ein Mechanismus eingebaut, der die ACT-LED auf Herzschlag-Blinken setzt, sobald das System fertig in `ENV1` gebootet ist – damit man weiß, wann man den Hardware-Boot-Umschalter wieder loslassen/zurückstellen kann.

autostart.sh – 2/2

- Hier im „example“-Branch installiert/aktiviert sie anschließend nur noch das Overlay-Dateisystem, und konfiguriert es so, dass /data eben kein Overlay, sondern dauerhaft schreibbar ist. In anderen Branches wird vor der Aktivierung des Overlay-Dateisystems z.B. auch noch weitere Software installiert, der Hardware-Watchdog aktiviert und/oder alle Ethernet-Interfaces in die Bridge aufgenommen.
- Zum Abschluss wird in ENV1 der Apt-Cache geleert, der Bootloader auf ENV2 gesetzt, und ein Neustart ausgelöst.
- Nun wiederholt sich das Spiel für ENV2 und ENV3 – wobei hier dann z.B. noch Konfigurationsdateien für die Dienste geschrieben werden und die vorher heruntergeladenen Pakete pro ENV installiert werden – im Example-Branch also „s1“.
- Am Ende der Installation von ENV3 wird dann der Block aus der autostart.sh gelöscht, der nur für die Installationen notwendig war, der Bootloader auf ENV2 gesetzt, ein Reboot ausgelöst, und wir sind in Produktion.

Besonderheiten bei x2go*

- x2goserver erstellt (wenn noch nicht vorhanden):
 - `rrpi-base-install/custom/x2goserver/`
 - Darin wiederum die Unterordner `user1/` und `user2/`
 - Und in diesen dann SSH-Schlüsselpaare `id_ed25519 / id_ed25519.pub`
- Die öffentlichen Schlüssel werden dann ins installierte x2goserver-System kopiert
- Da `/data` in diesem Branch „sync“ gemountet ist, wird das im Betrieb eher kein Performance-Wunder.
Entfernen des Parameters auf eigene Gefahr – denkt daran, auch `autostart.sh` liegt auf `/data`. Es ist zwar sichergestellt, dass das System nicht stehen bleibt, wenn es die Datei nicht findet (weil `/data` sich aufgrund von Fehlern nicht mounten lässt oder weil ihr die Datei gelöscht habt), aber so ganz unkaputtbar ist das System dann halt nicht mehr.
- x2gominiclient erstellt (wenn noch nicht vorhanden):
 - `rrpi-base-install/custom/x2goclient/`
 - Darin die Config-Dateien `printing`, `sessions`, `settings`, `.xsession` (landen im installierten x2gominiclient-System in `/home/[DefaultUser]/.x2goclient/` - `[DefaultUser]` ist, was ihr im Imager/in cloud-init als Benutzername angegeben habt)
 - Sowie ein angepasstes Startscript `x2goclient` (landet im installierten x2gominiclient-System dann in `/usr/local/bin/`)
- Die privaten Schlüssel aus `rrpi-base-install/custom/x2goserver/` landen im installierten x2gominiclient-System in `/home/[DefaultUser]/.ssh/user{1,2}/` → bei abweichenden Usernamen muss man evtl. noch die `autostart.sh` anpassen
- Daher: Wenn man x2goserver und x2gominiclient installieren will, dann immer zuerst x2goserver, damit x2gominiclient die Keys findet!
- Man kann die Dateien unterhalb von `rrpi-base-install/custom/` natürlich auch vorab manuell anlegen!
→ So kann man seine eigenen, angepassten Versionen einspielen und die Reihenfolge der Installation ist egal.

Besonderheiten bei *-printserver

- single-port-printserver ist „Fire & Forget“, aber hat keine festen Portzuweisungen, es gibt nur einen TCP-Port
- im installierten multi-port-printserver steckt die „Magie“ für die festen Portzuweisungen in:
 - /etc/udev/rules.d/050_persistent_printer_mappings.rules (Mapping erfolgt anhand bekannter USB/PCIe Port-IDs)
 - /dev/persistent_lp/
- Portzuweisungen:

Pi1:	Pi3B+:
ETH lp0	ETH lp2 lp1
PRT lp1	PRT lp3 lp0

Pi3B:	Pi4B:
ETH lp0 lp2	lp0 lp2 ETH
PRT lp1 lp3	lp1 lp3 PRT

Besonderheiten bei docker-mailcow

- Die `autostart.sh` im `docker-mailcow`-Branch hat folgende Besonderheiten:
 - Sie setzt `bind-mounts` für `docker` und `mailcow`, so dass `/opt`, `/var/lib/containerd` und `/var/lib/docker` in Wirklichkeit auf `/data/ENV2/` (bzw. `/data/ENV3/`) statt auf `/` liegen und somit persistent schreibbar sind
 - Sie holt `docker-mailcow` per `git clone`, nicht per `apt-get`, braucht also anders als die anderen Branches auch in dem Teil der Installation, der erst beim ersten Boot auf dem Raspi selbst läuft, immer noch Internet
- Man könnte auf die Idee kommen, zu sagen, `ENV2` und `ENV3` sollen sich die Mailcow-Daten teilen, indem man einen weiteren `Bind-Mount` dafür anlegt.
 - Das hat aber das Potential, heftig schiefzugehen, weil ja auch eine Datenbank und nicht nur ein Dateisystem im Spiel ist.
 - Daher zum Abgleich zwischen den Instanzen in `ENV2` und `ENV3` vor dem Upgrade besser mit Mailcows Cold-Standby/Rolling Backup-Strategie arbeiten, siehe: https://docs.mailcow.email/backup_restore/b_n_r-coldstandby (Abschnitt „You should know“ beachten) → erst mal alles in ein anderes Verzeichnis schieben, als Backup, und dann von dort im anderen ENV im gebooteten Zustand einspielen.
- Da `/data` auch in diesem Branch „`sync`“ gemountet ist, wird auch das im Betrieb eher kein Performance-Wunder. Entfernen des Parameters auf eigene Gefahr, wie beim `x2goserver`-Branch erklärt.

Wie umschalten/updates?

- Im Git-Repo gibt es den Ordner `./usr/local/sbin`
- Die darin befindlichen Skripte sind nach der Installation in allen ENVs in `/usr/local/sbin` zu finden:
 - Das Skript `update-other-env.sh` ist die empfohlene Methode, um ein aktuell nicht gebootetes ENV „offline“ per chroot upzudaten. Die Nummer des gewünschten Ziel-ENVs ist als Parameter zu übergeben. Also aus ENV2 heraus für ENV3: „`update-other-env.sh 3`“
 - Der Bootloader wird danach NICHT automatisch neu gesetzt. Dafür ist das Skript `permanent-boot-switch.sh` zuständig, welches separat aufzurufen ist. Auch hier ist der erste Parameter wieder die Nummer des zukünftig zu startenden ENVs. Als zweiter Parameter kann „reboot“ angegeben werden, dann führt das Skript direkt nach der Änderung den Reboot durch. Dieser Change ist permanent (bis zur nächsten Änderung). Möchte man nur einmalig in eine andere Umgebung booten, reicht der Aufruf „`sudo reboot n`“ – mit einer Ziffer von 1 bis 3 statt n.

Experten-Skripte

- Liegen ebenfalls im Git-Repo im Ordner `./usr/local/sbin`
- Sind nach der Installation ebenfalls in allen ENVs in `/usr/local/sbin` zu finden:
 - `disable-overlay.sh` – deaktiviert das Overlayroot für das aktuell gebootete ENV. Wirkt beim nächsten Neustart. Sofern `/boot/firmware` ebenfalls readonly gemountet war, stellt es den Schreibzugriff darauf wieder her, braucht dann aber einen Reboot und einen weiteren Aufruf sowie einen zweiten Reboot zur kompletten Deaktivierung.
 - `enable-overlay-no-recurse.sh` – reaktiviert das Overlayroot beim nächsten Neustart wieder, spart aber `/boot/firmware` (außer, man erzwingt es über den Parameter „enable-bootro“) und `/data` aus.
 - `enable-overlay.sh` – analog oben, aber spart `/data` **NICHT** aus. Nicht empfohlen.
- Werden im Normalfall nicht benötigt

Nächste X2Go-Events

Noch nicht angemeldet/noch keine Zusage:

- 17.09.-19.09.2026 KieLux (vermutl. nur Onlinevortrag)
- 26.9.2026 LinuxDay.AT, Dornbirn

Stand und Vortrag:

- 12.11.2026 IT-Kongress, Hochschule Neu-Ulm

Termin/Ort noch offen:

- „X2Go: The Gathering 2026“
- orca e.V. Jahreshauptversammlung 2026

Wiki: <https://wiki.x2go.org/doku.php/events:start>



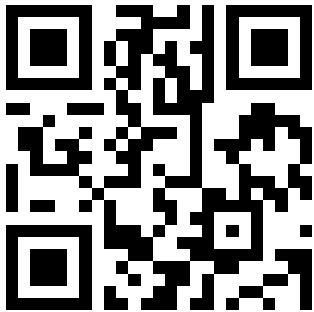
Event-
Wikiseite

X2Go lebt vom Mitmachen

- Helfer gesucht!
- X2Go kann immer zwei Dinge von euch brauchen:
 - Zeit/KnowHow – auch von Nicht-Programmierern!
 - Geld/Hardware/Dienstleistung: Man kann ...
 - über den orca e.V. (gemeinnützig) eine zweckgebundene Spende an X2Go leisten
 - eine der Firmen im Projekt mit einer konkreten Aufgabe (Bugfix, Feature Request) beauftragen

Spenden/Aufträge

- Für Spenden haben wir mehrere WirWunder/Betterplace-Seiten (Events, Infrastrukturkosten, ...) direkt auf der Wiki-Startseite: <https://wiki.x2go.org>
- Firmen, die für Aufträge zur Verfügung stehen: <https://wiki.x2go.org/doku.php/0spnn5> (null-spnn-fünf)

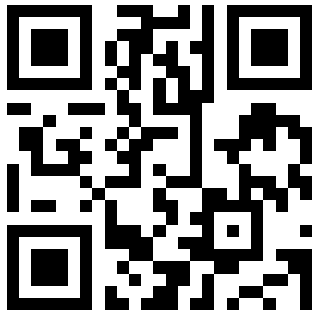


Spendenlink



Liste der Supportfirmen

Vielen Dank für das Interesse!
Mehr Infos und Q&A am Stand!



Spendenlink



Liste der Supportfirmen